

# FPGA Design & Simulation: Sequential Logic Analysis

From Single-Bit Flip-Flops to Synchronous Counters

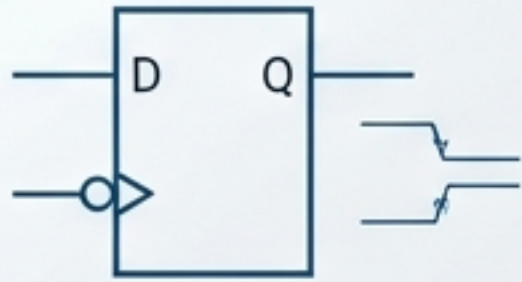


Context: Technical deep-dive based on the FPGA Design Exercise by Mahi P. Doshi.



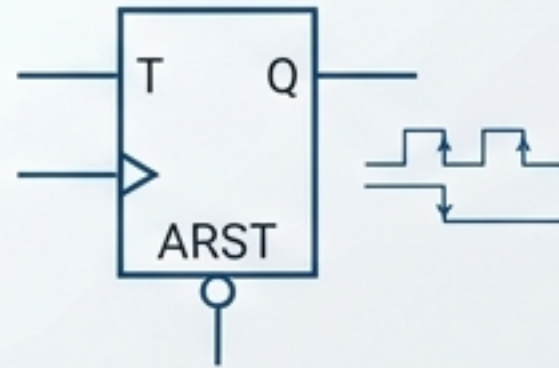
# The Logic Ladder: From Storage to Sequencing

## Capture



D Flip-Flop  
(Negative Edge)

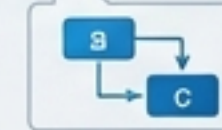
## Control



T Flip-Flop  
(Async Reset)

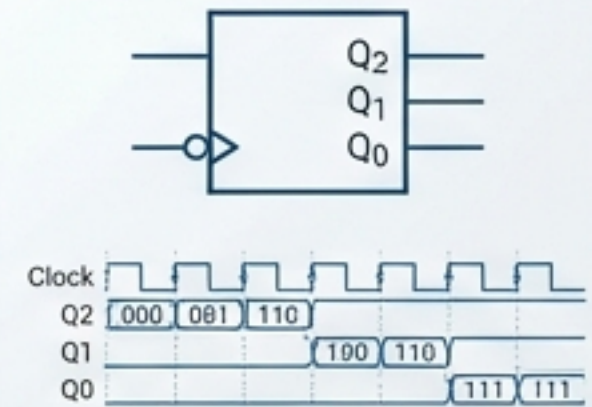
## Order

```
always @(posedge clk) begin
  a = b + 1;
  c = a << 2;
end
```



Blocking  
Assignments

## Integration



Synchronous  
3-Bit Counter

We validate correct hardware behavior by analyzing code alongside simulation waveforms, verifying that logic translates into reality.



# Precision Data Capture: The Negative Edge D Flip-Flop

## The Code

```
`timescale 1ns/1ps
module d_ff_neg (
    input clk,
    input d,
    output reg q
);
    always @(negedge clk)
        q <= d;
endmodule
```

## The Reality



The 'Photographer' Circuit: Updates to output 'q' occur strictly on the falling edge of the clock. Stability is maintained between edges.

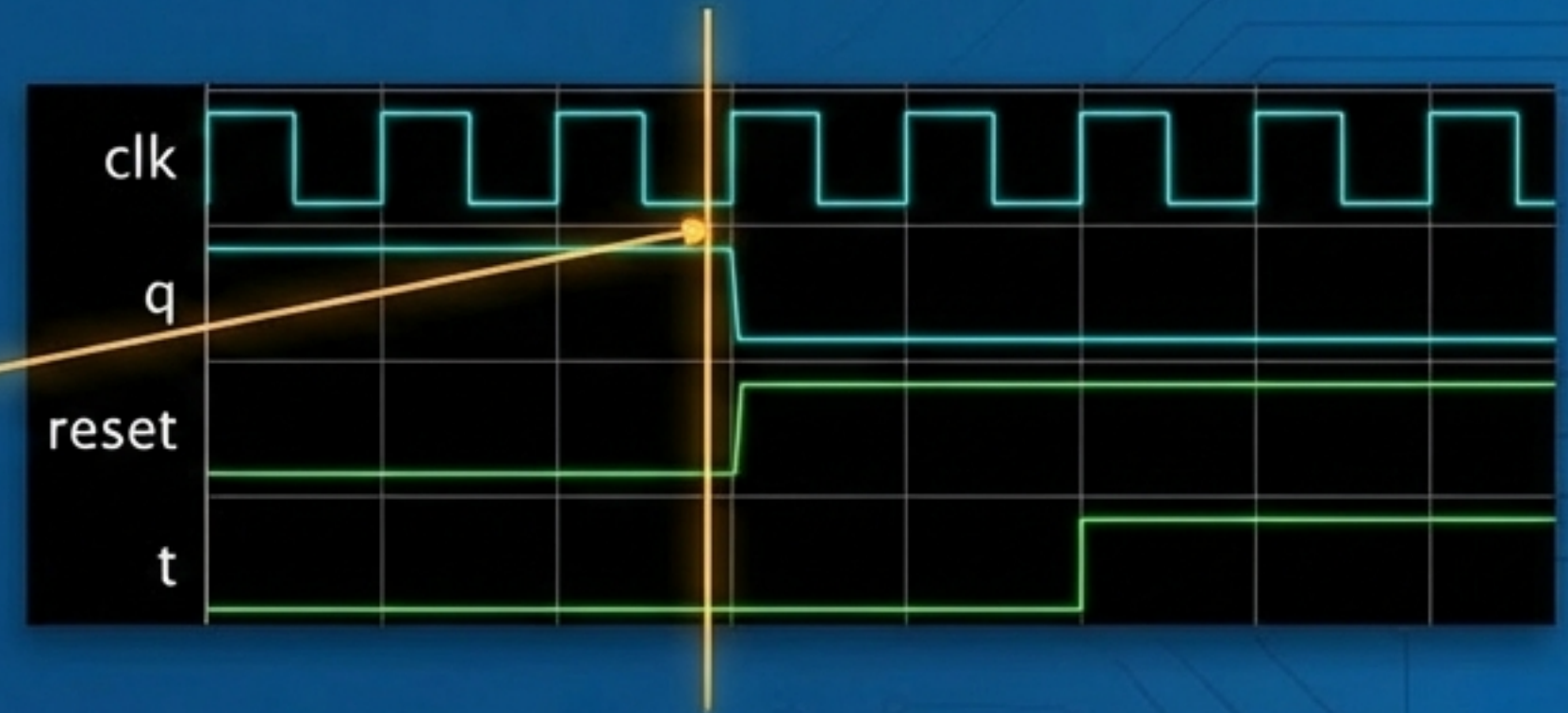


# Asynchronous Control: The T Flip-Flop

## The Code

```
module t_ff_async (  
    input clk,  
    input reset,  
    input t,  
    output reg q  
);  
  
always @(posedge clk or posedge reset)  
begin  
    if (reset)  
        q <= 0;  
    else if (t)  
        q <= ~q;  
end  
endmodule
```

## The Reality



**Priority Logic:** The asynchronous reset forces the output to 0 immediately, overriding the clock signal. This ensures the system can always be initialized to a known state.

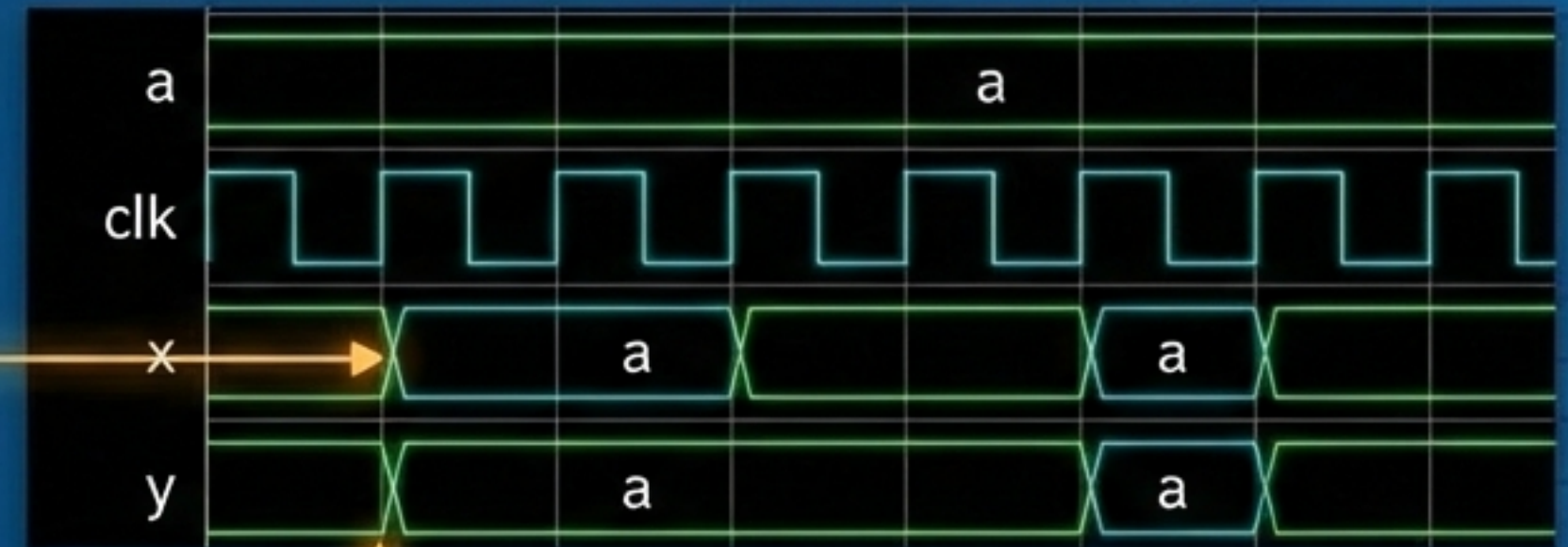


# Execution Order: The Mechanics of Blocking Assignments

## The Code

```
module blocking_example (  
    input clk,  
    input a,  
    output reg x,  
    output reg y  
);  
  
always @(posedge clk)  
begin  
    x = a;  
    y = x;  
end  
endmodule
```

## The Reality



**Sequential Execution:** In a blocking assignment (=), the second statement sees the newly updated value of x immediately. This mimics software-like progression within a single clock cycle.



# System Integration: The 3-Bit Synchronous Up Counter

```
module up_counter (  
    input clk,  
    input reset,  
    output reg [2:0] q  
);  
  
always @(posedge clk or posedge reset)  
begin  
    if (reset)  
        q <= 3'b000;  
    else  
        q <= q + 1;  
end  
endmodule
```

3-Bit Register Bus

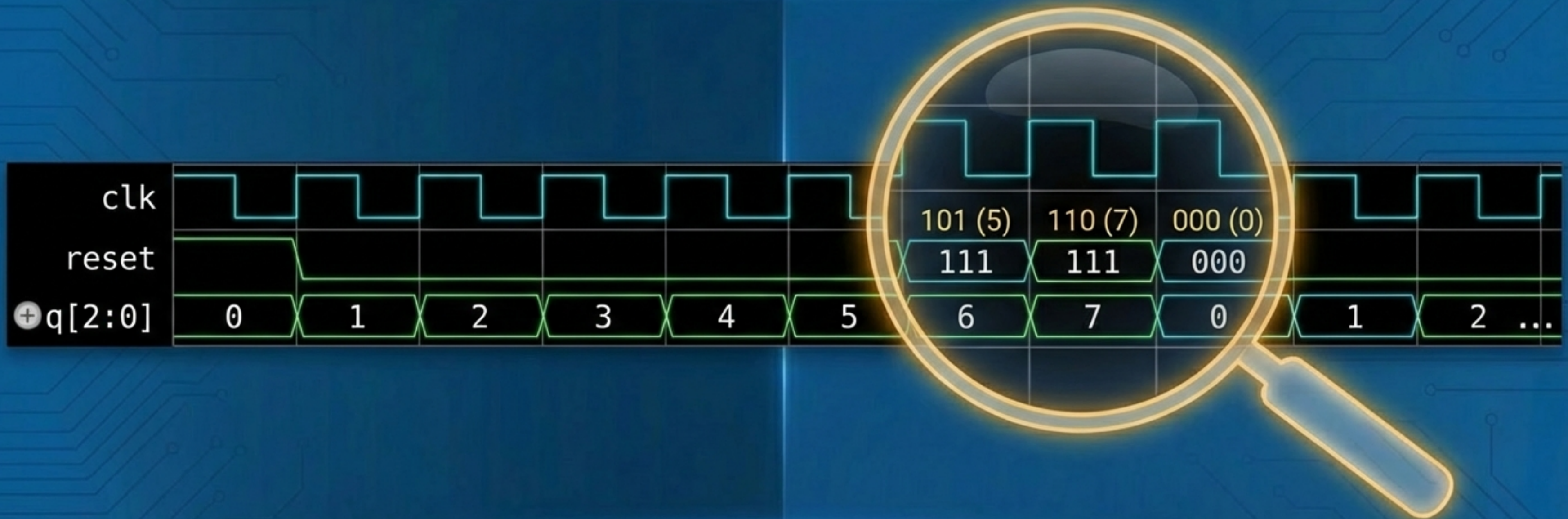
Synchronous Trigger

Accumulation Logic

**The Accumulator:** All three bits update simultaneously on the rising clock edge. This minimizes propagation delay compared to asynchronous ripple counters.



# Verification: Visualizing the Binary Sequence



**Cycle Complete:** The counter progresses linearly from 0 to 7. Upon reaching the maximum 3-bit value, it naturally rolls over to 0 on the next clock edge.



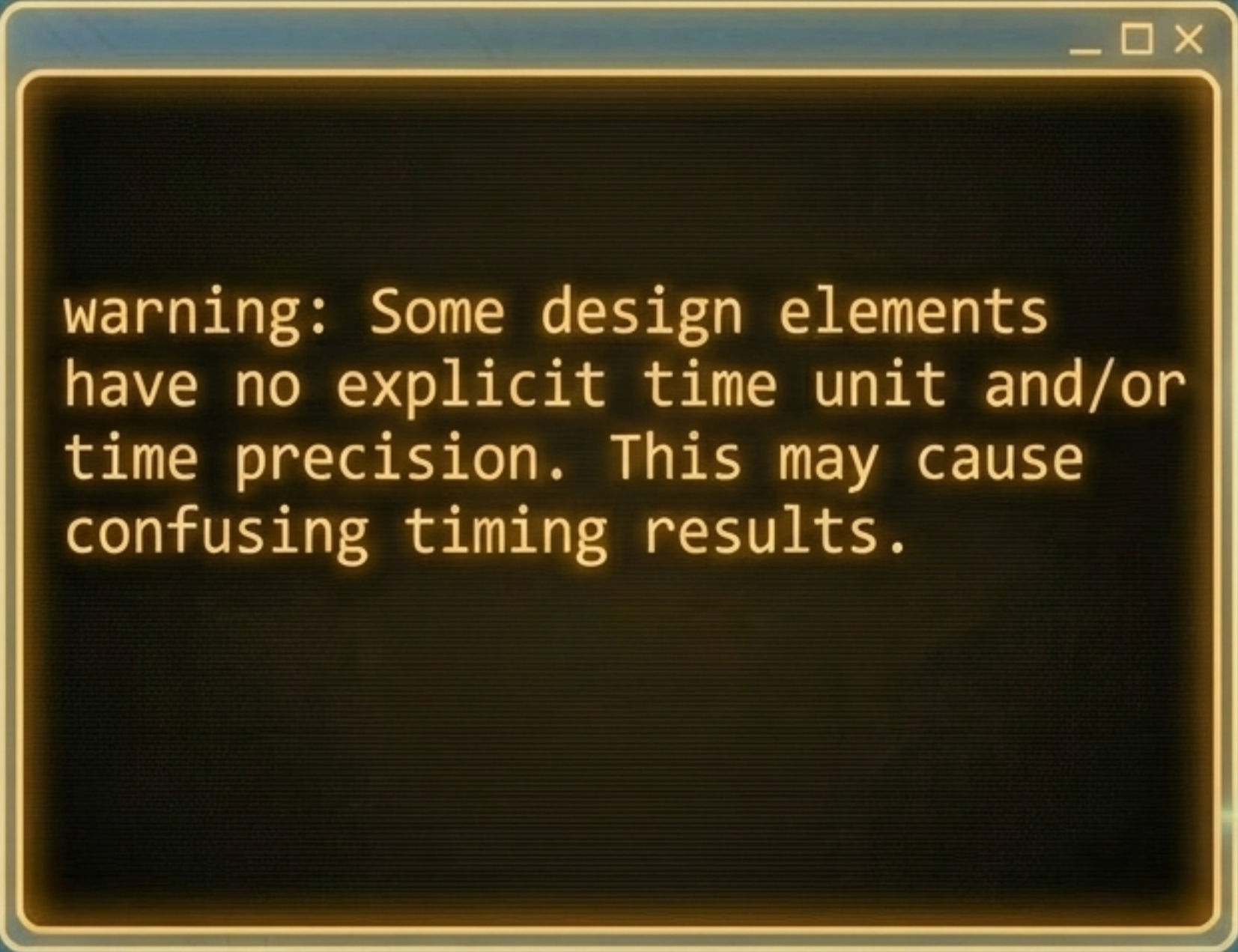
# Component Architecture: Selecting the Right Tool

| D Flip-Flop                                                                                 | T Flip-Flop                                                                                  | Synchronous Counter                                                                              |
|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <p>Role: Data Capture.</p> <p>Best For: Timing-sensitive snapshots and pipeline stages.</p> | <p>Role: State Toggling.</p> <p>Best For: Frequency division and simple on/off switches.</p> | <p>Role: Sequencing.</p> <p>Best For: Event counting, addressing memory, and timing control.</p> |

**Successful FPGA architecture relies on combining these primitives to manage state and time.**



# Engineer's Log: Timing Precision & Design Warnings



```
warning: Some design elements  
have no explicit time unit and/or  
time precision. This may cause  
confusing timing results.
```

## Critical Insight: The ``timescale`` Directive

While the source code defines ``timescale 1ns/1ps``, the compiler warns of potential precision mismatches. In real-world hardware, undefined time units can lead to critical simulation failures. Explicitly defining time scales is mandatory for verification.



# Final Validation



- Verified negative-edge data capture (D-FF)
- Validated asynchronous reset priority (T-FF)
- Demonstrated sequential execution flow (Blocking Assignment)
- Confirmed synchronous state accumulation (3-Bit Counter)

**The logic holds. The design is ready for synthesis.**